

Storm Generator v2.0

Implementation Specification

Thames Valley Physical Risk Swaps - Enhanced Multi-Storm Sequences

Project: PRS Platform Storm Generator v2.0

Date: March 5, 2026

Prepared for: David Kelly

Scope: Phase 1 (Extended Durations) + Phase 2 (Storm Sequences with Drainage Windows)

Version: 2.0-multi-storm

Executive Summary

This specification provides detailed implementation guidance for a focused storm generation system targeting **single-scenario, two-storm (doublet) sequences** with explicit inter-storm drainage periods to model compound flooding events in the Thames Valley.

Key Innovation

Moving from independent storm sampling to **temporally-correlated multi-storm sequences** where:

- Each scenario contains **1-5 precipitation events** (isolated, doublet, cluster, or persistent patterns)
- Clear **inter-storm gaps** (6–144 hours depending on type) allow observable river level decline
- All precipitation stops **≥12 hours before hour 168** to show final drainage
- Antecedent soil moisture from Storm 1 amplifies Storm 2 response
- Hydraulic and hydrologic models work together to capture compounding effects

Success Criteria

- Generate 10,000 realistic two-storm flood scenarios
 - Respect 168-hour insurance event windows
 - Show clear water level decline between precipitation pulses
 - Model soil moisture persistence and compounding effects
 - Maintain market-calibrated intensity distribution (20-60 bps fair spreads)
 - File sizes remain manageable (<50 MB for 10K events)
 - Performance: <2 minutes to generate 10K doublet scenarios with gauge responses
-

1. Project Context and Constraints

1.1 Document Purpose

This is a **specification document**, not implementation code. It describes:

- What needs to be built
- How components should interact
- Data structures and schemas
- Algorithms and logic flows
- Integration points with existing code
- Testing and validation criteria
- Mathematical foundations for hydrologic-hydraulic coupling

1.2 Project Constraints

Scenario structure: Four sequence types

(isolated/doublet/cluster/persistent) with 1-5 storms per sequence

Event count: 10,000 doublet scenarios maximum

Event window: 168 hours (7 days) per insurance industry standard

Drainage requirement: Final 12 hours (156–168h) must be precipitation-free

Gauge data: DEFRA gauges are real, others extrapolated

Climate scenarios: NOT in scope (present-day conditions only)

Backward compatibility: Must maintain ability to load existing storm files

1.3 Known Limitations

Validation data: Limited historical gauge data availability for doublet sequences

Antecedent conditions: Initial implementation uses simplified state model; full state-space model (Section 6) is future enhancement

Tidal influence: Current gauge levels may show tidal undulations; tide schedule injection capability to be added if needed

2. Core Architecture: Multi-Storm Sequences

2.1 Overview

Each "scenario" is a **StormSequence** object containing 1-5 **StormScenario** objects (storms), where multiple storms are separated by inter-storm gaps that allow partial drainage. The entire sequence fits within a 168-hour event window, with all precipitation ending by hour 156 to allow final drainage observation.

2.1.1 Sequence Type Definitions

Type	Storm Count	Total Duration	Inter-storm Gaps	Use Case
isolated	1	6-240 hours	N/A	Single storm
doublet	2	48-156 hours	12-72 hours	Two storms, short gap
cluster	3-4	96-156 hours	24-96 hours	Multiple storms
persistent	4-5	96-156 hours	12-72 hours	Persistent wet pattern

Table 1: Sequence type definitions

2.2 Data Structures

2.2.1 StormScenario (Individual Storm)

```
@dataclass
```

```
class StormScenario:
```

```
    """
```

```
A single precipitation event within a sequence.
```

```
    """
```

```
# Identity
```

```
storm_id: str # Format: "STORM-{uuid4}"
```

```
scenario_id: str # Parent sequence ID
```

```
# Temporal
```

```
start_time: str # ISO format, hours from sequence start
```

```
duration_hours: int # Storm duration
```

```
# Intensity
```

```
intensity_category: str # 'moderate', 'severe', 'extreme', 'catastrophic'
```

```
intensity_factor: float # Multiplier for base intensity
```

```
precipitation_mm: float # Total precipitation
```

```
# Metadata
```

```
generated_at: str
```

```
catchment_id: str
```

2.2.2 StormSequence (Multi-Storm Container)

```
@dataclass
```

```
class StormSequence:
```

```
    """
```

```
A sequence of 1-5 related storm events treated as one flood event.
```

```
Insurance industry 'hours clause' implementation.
```

```
    """
```

```
# Identity
```

```
sequence_id: str # Format: "SEQ-{uuid4}"
```

```
sequence_type: str # 'isolated', 'doublet', 'cluster', 'persistent'
```

```

# Temporal structure
sequence_start: str          # ISO format
sequence_end: str            # Must be ≤ 156 hours from start
total_duration_hours: int    # Actual duration of precip + gaps
event_window_hours: int      # Always 168
drainage_window_hours: int   # Final dry period (≥12 hours)

# Storm composition
storms: List[StormScenario]  # 1-5 storms
num_storms: int              # 1-5

# Temporal gaps
inter_storm_gaps_hours: List[int] # Gaps between consecutive storms

# Aggregate statistics
total_precipitation_mm: float
max_intensity_mm: float
avg_intensity_mm: float
cumulative_intensity_factor: float

# Antecedent conditions (for gauge response)
antecedent_soil_moisture: Optional[str] = None    # 'dry', 'normal', 'wet'
antecedent_groundwater: Optional[str] = None     # 'low', 'normal', 'high'

# Metadata
generated_at: str
catchment_id: str

```

Validation rules:

- `num_storms` must equal `len(storms)`
- `storms` list: minimum 1, maximum 5
- `sequence_type` in {'isolated', 'doublet', 'cluster', 'persistent'}
- `len(inter_storm_gaps_hours)` must equal `num_storms - 1`
- `sequence_end` ≤ 156 hours (leaves ≥12h drainage)
- `total_duration_hours` = `sum(storm durations)` + `sum(inter_storm_gaps_hours)`

- Temporal ordering: `storms[i].start_time < storms[i+1].start_time`
-

3. Storm Generation Parameters

3.1 Duration Parameters (Phase 1 - Extended Durations)

Updated from original spec to support up to 240-hour maximum durations for individual storms within sequences.

```
DURATION_PARAMS = {  
# (min, mode, max) hours - triangular distribution  
'moderate': (12, 30, 60), # 2.5 day maximum  
'severe': (24, 48, 96), # 4 day maximum  
'extreme': (36, 72, 144), # 6 day maximum  
'catastrophic': (48, 120, 240), # 10 day maximum  
}
```

Key design decisions:

1. Triangular distribution preserved - maintains statistical properties
2. Each tier approximately doubles previous maximum
3. Mode shifted up - reflects that longer events are more common than previously assumed
4. Conservative extreme tail - catastrophic events get full 10-day capability

3.2 Inter-Storm Gap Parameters

Gaps must be long enough to show observable river level decline between storms.

```
INTER_STORM_GAP_PARAMS = {  
# (min, mode, max) hours - triangular distribution  
'short': (6, 18, 36), # Minimal drainage  
'medium': (24, 48, 72), # Partial drainage  
'long': (48, 96, 144), # Substantial drainage  
}
```

```
GAP_TYPE_BY_SEQUENCE = {  
'doublet': 'medium',
```

```
'cluster': 'medium',  
'persistent': 'short'  
}
```

Gap type selection: Gap type varies by sequence type to create different drainage patterns.

Physical interpretation:

- **Medium gaps (24-72h):** River levels drop 30-60% from Storm 1 peak before Storm 2 begins
- **Long gaps (48-144h):** River levels drop 60-80% from Storm 1 peak, approaching baseline

3.3 Intensity Parameters

Storm intensities vary around base intensity to create realistic variability within sequences.

INTENSITY_VARIATION = 0.20 # $\pm 20\%$ around base intensity

Probability that first storm is most intense

FIRST_STORM_DOMINANT_PROB = 0.30 # 30%

Otherwise storms maintain or increase intensity (70%)

Rationale: In compound flooding, later storms often produce higher peaks due to antecedent wetness, even if precipitation intensity is similar or slightly lower. For persistent sequences (4-5 storms), intensities can vary at each pulse while maintaining the wet pattern.

3.4 Drainage Window Constraint

```
EVENT_WINDOW_HOURS = 168
MIN_DRAINAGE_WINDOW_HOURS = 12 # Precipitation-free period at
end
```

**Constraint: $\text{sequence_end} \leq$
 $\text{EVENT_WINDOW_HOURS} -$
 $\text{MIN_DRAINAGE_WINDOW_HOURS}$**

```
MAX_PRECIP_END_HOUR = 156
```

4. Multi-Storm Sequence Generation Algorithm

4.1 High-Level Flow

1. Sample intensity category for the sequence
2. Decide: isolated or multi-storm sequence?
3. If multi-storm: Select type (doublet/cluster/persistent)
4. Sample sequence duration constraint
5. Generate individual storms with time offsets
6. Sample inter-storm gaps appropriate to sequence type
7. Verify timing constraint: final storm end $\leq$ hour 156
8. If constraint violated, resample durations/gaps until satisfied
9. Create StormSequence object
10. Calculate aggregate statistics

4.2 Detailed Algorithm

```
def generate_sequence(intensity_category: str,
catchment_id: str,
sequence_type: str = None) -> StormSequence:
    """
```

Generate a storm sequence with 1-5 storms.

Args:

intensity_category: Storm intensity level

catchment_id: Catchment identifier

sequence_type: Force specific type, or None to sample probabilistically

Returns:

StormSequence with 1-5 storms, ending by hour 156

"""

```
sequence_id = f"SEQ-{uuid4()}"
```

```
# Step 1: Determine sequence type if not specified
```

```
if sequence_type is None:
```

```
    # Decide isolated vs multi-storm based on intensity
```

```
    if should_generate_sequence(intensity_category):
```

```
        sequence_type = sample_sequence_type(intensity_category)
```

```
    else:
```

```
        sequence_type = 'isolated'
```

```
# Step 2: Determine number of storms
```

```
num_storms = get_storm_count(sequence_type)
```

```
# Step 3: Sample base intensity
```

```
base_intensity = sample_intensity_factor(intensity_category)
```

```
# Step 4: Generate each storm
```

```
storms = []
```

```
current_time = 0
```

```
gaps = []
```

```
for i in range(num_storms):
```

```
    # Sample duration
```

```
    duration = sample_duration(intensity_category)
```

```
    # Sample intensity (with variation and tendency to increase)
```

```
    if i == 0 or random() < FIRST_STORM_DOMINANT_PROB:
```

```
        intensity = base_intensity * (1 + uniform(-0.2, 0.0))
```

```
    else:
```

```

        intensity = base_intensity * (1 + uniform(0.0, 0.2))

storm_1 = StormScenario(
    storm_id=f"STORM-{uuid4()}",
    scenario_id=sequence_id,
    start_time="0h", # Sequence starts at hour 0
    duration_hours=duration_1,
    intensity_category=intensity_category,
    intensity_factor=intensity_1,
    precipitation_mm=calculate_precipitation(duration_1, intensity_1),
    generated_at=datetime.now().isoformat(),
    catchment_id=catchment_id
)

# Step 3: Sample inter-storm gap
gap_hours = sample_gap(gap_type)

# Step 4: Generate Storm 2
storm_2_start = duration_1 + gap_hours
duration_2 = sample_duration(intensity_category)

# Intensity correlation: 70% chance Storm 2 ≥ Storm 1
if random() < 0.70:
    intensity_2 = base_intensity * (1 + uniform(0.0, 0.2))
else:
    intensity_2 = base_intensity * (1 + uniform(-0.2, 0.0))

storm_2 = StormScenario(
    storm_id=f"STORM-{uuid4()}",
    scenario_id=sequence_id,
    start_time=f"{storm_2_start}h",
    duration_hours=duration_2,
    intensity_category=intensity_category,
    intensity_factor=intensity_2,
    precipitation_mm=calculate_precipitation(duration_2, intensity_2),
    generated_at=datetime.now().isoformat(),
    catchment_id=catchment_id
)

```

```

# Step 5: Verify timing constraint
storm_2_end = storm_2_start + duration_2

if storm_2_end > MAX_PRECIP_END_HOUR:
    # Resample with shorter durations or gap
    return generate_doublet_sequence(
        intensity_category, catchment_id, gap_type
    )

# Step 6: Create sequence object
sequence = StormSequence(
    sequence_id=sequence_id,
    sequence_type='doublet',
    sequence_start=datetime.now().isoformat(),
    sequence_end=f"{storm_2_end}h",
    total_duration_hours=storm_2_end,
    event_window_hours=168,
    drainage_window_hours=168 - storm_2_end,
    storms=[storm_1, storm_2],
    num_storms=2,
    inter_storm_gap_hours=gap_hours,
    total_precipitation_mm=storm_1.precipitation_mm + storm_2.precipitation_mm,
    max_intensity_mm=max(storm_1.intensity_factor, storm_2.intensity_factor),
    avg_intensity_mm=(storm_1.intensity_factor + storm_2.intensity_factor) / 2,
    cumulative_intensity_factor=storm_1.intensity_factor + storm_2.intensity_factor,
    antecedent_soil_moisture='normal', # Initial state
    antecedent_groundwater='normal',
    generated_at=datetime.now().isoformat(),
    catchment_id=catchment_id
)

return sequence

```

4.3 Sequence Probability Parameters

```
SEQUENCE_PROBABILITY = {  
# Probability of multi-storm sequence by intensity  
'minimal': 0.05, # 5% - rarely cluster  
'baseline': 0.15, # 15%  
'moderate': 0.30, # 30%  
'severe': 0.50, # 50%  
'extreme': 0.70, # 70%  
'catastrophic': 0.85 # 85% - almost always sequences  
}
```

```
SEQUENCE_TYPE_WEIGHTS = {  
# [doublet, cluster, persistent] weights by intensity  
'moderate': [0.50, 0.35, 0.15],  
'severe': [0.30, 0.50, 0.20],  
'extreme': [0.20, 0.45, 0.35],  
'catastrophic': [0.15, 0.45, 0.40]  
}
```

```
def get_storm_count(sequence_type: str) -> int:  
    """Return number of storms for sequence type."""  
    counts = {  
        'isolated': 1,  
        'doublet': 2,  
        'cluster': choice([3, 4]),  
        'persistent': choice([4, 5])  
    }  
    return counts[sequence_type]
```

4.4 Batch Generation

```
def generate_event_set(count: int = 10000,  
    intensity_weights: dict = None,  
    force_sequence_type: str = None) -> List[StormSequence]:  
    """
```

Generate a set of storm sequences.

Args:

count: Number of sequences to generate

```
intensity_weights: Dict of {category: probability}
force_sequence_type: Force all sequences to specific type, or None for prob
```

Returns:

List of StormSequence objects

```
"""
if intensity_weights is None:
    intensity_weights = {
        'moderate': 0.40,
        'severe': 0.35,
        'extreme': 0.20,
        'catastrophic': 0.05
    }

sequences = []
for i in range(count):
    category = sample_intensity_category(intensity_weights)
    seq = generate_doublet_sequence(
        intensity_category=category,
        catchment_id='thames',
        gap_type=gap_type
    )
    sequences.append(seq)

return sequences
```

5. Hydrologic-Hydraulic Integration

5.1 Gauge Response Model

The gauge response model (stormgauge.py) transforms precipitation time series into river level (or flow) hydrographs at each gauge location.

Key capabilities:

- Time-series based convolution with lag and decay parameters
- Handles arbitrary storm durations (tested up to 240 hours)

- Configurable response kernels per gauge/catchment
- Supports antecedent moisture state amplification

5.2 Compounding Between Storms

For doublet sequences, gauge responses must capture compounding:

Storm 1:

- Rises from baseline river level
- Peak occurs at `storm_1_end + lag_hours`
- Begins recession during gap period

Gap period:

- Partial drainage (river level drops but does NOT return to baseline)
- Soil moisture remains elevated
- Groundwater storage decays slowly

Storm 2:

- Rises from elevated baseline (not from original baseline)
- Produces higher peak than Storm 1 of equal intensity would produce
- Final recession extends into drainage window (hours 156–168)

Implementation approach:

```
def compute_sequence_gauge_response(sequence: StormSequence,
gauge_id: str,
gauge_params: dict) -> np.ndarray:
    """
```

Compute continuous gauge response for entire doublet sequence.

Returns:

Array of length 168 (hourly levels for full event window)

"""

Initialize state

soil_moisture = initial_soil_moisture()

groundwater = initial_groundwater()

```

river_level = baseline_level(gauge_id)

levels = np.zeros(168)

for hour in range(168):
    # Check if precipitation is occurring
    precip = get_precipitation_at_hour(sequence, hour)

    # Update hydrologic state
    soil_moisture, groundwater, quickflow = update_hydrology(
        soil_moisture, groundwater, precip, hour
    )

    # Update river level (hydraulic routing)
    river_level = update_hydraulics(
        river_level, quickflow, gauge_params
    )

    levels[hour] = river_level

return levels

```

5.3 Antecedent Conditions State Model

Simplified state evolution for initial implementation (see Section 6 for full mathematical model):

Soil moisture evolution:

$$S_{t+1} = S_t + \alpha_P \cdot P_t - \beta_S \cdot S_t \quad (1)$$

where:

- S_t = soil moisture at hour t
- P_t = precipitation at hour t
- α_P = infiltration coefficient (0.6–0.8)
- β_S = drainage coefficient (0.01–0.05 per hour)

Groundwater evolution:

$$G_{t+1} = G_t + \beta_S \cdot S_t - \gamma_G \cdot G_t \quad (2)$$

where:

- G_t = groundwater storage at hour t
- γ_G = baseflow coefficient (0.005–0.02 per hour)

Runoff amplification:

$$Q_t = (1 + \lambda_S \cdot S_t + \lambda_G \cdot G_t) \cdot P_t \quad (3)$$

where:

- Q_t = quickflow/runoff at hour t
- λ_S, λ_G = amplification coefficients (calibrated from gauge data)

Practical benefits:

- Models why Storm 2 produces higher peaks than Storm 1 of equal intensity
- Explains 168-hour event window (time for partial drainage)
- Provides mechanistic basis for gap duration effects
- Shows observable level decline during gaps

5.4 Tidal Signal Injection (Optional)

If observed gauge levels show tidal undulations not captured by the hydrologic model, add a separate tidal component:

River-only level: $R(t)$ from storm generator and gauge response model

Tidal level: $T(t)$ from external tide predictions or parametric model

Total gauge level: $L(t) = R(t) + \alpha \cdot T(t)$

where α is a scaling factor (0–1) based on gauge distance from estuary.

Tidal model options:

1. **Data-driven:** Use actual tide predictions for London Bridge, interpolate and repeat for 168-hour windows
2. **Analytical:** Approximate semi-diurnal tide with sum of sinusoids (M2, S2 components)

Implementation note: Compute drainage on $R(t)$ alone (hydrologic recession), then add $T(t)$ afterwards so tides modulate observed level without affecting catchment storage dynamics.

6. Mathematical Foundations

6.1 State-Space Model for Antecedent Conditions (Future Enhancement)

For a more rigorous treatment of compounding effects, implement a full state-space model:

State evolution:

$$S_{t+1} = f_S(S_t, G_t, P_t, \Delta t) + \epsilon_S \quad (4)$$

$$G_{t+1} = f_G(S_t, G_t, \Delta t) + \epsilon_G \quad (5)$$

where:

- f_S, f_G = nonlinear state transition functions
- ϵ_S, ϵ_G = stochastic components (process noise)

Observation model:

$$L_t = h(R_t, S_t, G_t) + \nu_t \quad (6)$$

where:

- h = observation function (rating curve)
- ν_t = measurement noise

Calibration approach:

1. Use historical Thames doublet events (2013/14, 2000, 2003 floods)
2. Fit state-space model using Extended Kalman Filter or Particle Filter
3. Estimate amplification coefficients λ_S, λ_G via maximum likelihood
4. Validate that synthetic doublets match observed peak ratios (Storm 2 / Storm 1)

6.2 Duration-Intensity Correlation

Physically, long-duration storms tend to be lower intensity (stratiform), while short convective storms are high intensity. Model this using a Gaussian copula with negative correlation:

$$\rho_{DI} = -0.3 \quad (7)$$

Practical sampling:

1. Sample (U_D, U_I) from Gaussian copula with correlation ρ_{DI}
2. Transform: $D = F_D^{-1}(U_D), I = F_I^{-1}(U_I)$
3. Result: Long storms $\rightarrow$ lower intensity, short storms $\rightarrow$ higher intensity

where F_D, F_I are triangular CDFs for duration and intensity.

6.3 Gap Duration and Drainage Efficiency

The relationship between gap duration and river level recovery can be modeled empirically:

$$\frac{L_{\text{gap_end}} - L_{\text{baseline}}}{L_{\text{storm1_peak}} - L_{\text{baseline}}} = \exp\left(-\frac{\text{gap_hours}}{\tau_{\text{recession}}}\right) \quad (8)$$

where $\tau_{\text{recession}}$ is the recession timescale (typically 24–72 hours for Thames catchment).

Calibration: Fit $\tau_{\text{recession}}$ from historical hydrographs to ensure modeled drainage matches observed behavior.

7. Implementation Architecture and Modularity

7.1 Code Structure Principles

CRITICAL REQUIREMENT: The storm generator, gauge response model, and related utilities must be implemented as highly modular components to support maintenance, testing, and future extensions.

Modularity guidelines:

- Functions should be cohesive and focused on a single responsibility
- **Target: ≤ 300 lines per module or function**
- **Hard maximum: 450 lines per module or function**
- Where logic exceeds this, decompose into smaller helpers

Example decomposition:

Instead of one large `generate_and_process_sequences()` function, split into:

- `sample_duration(category)` - 20 lines
- `sample_gap(gap_type)` - 15 lines
- `generate_storm(params)` - 50 lines
- `generate_doublet_sequence(params)` - 100 lines
- `compute_gauge_response(sequence, gauge)` - 80 lines
- `update_hydrology(state, precip)` - 60 lines
- `update_hydraulics(level, inflow)` - 40 lines

Benefits:

1. Easier debugging (isolate failures to specific functions)
2. Targeted unit tests (test `sample_gap` independently)
3. Reduced regression risk when adding enhancements
4. Clearer code review and approval process
5. Simpler integration of mathematical enhancements (Section 6)

7.2 Suggested Module Structure

```
port/src/
├── storm.py # Core StormScenario dataclass
├── storm_sequence.py # StormSequence dataclass
├── generators/
│   ├── duration_sampler.py # Sample storm durations
│   ├── gap_sampler.py # Sample inter-storm gaps
│   ├── intensity_sampler.py # Sample intensities
│   ├── doublet_generator.py # Generate doublet sequences
│   └── batch_generator.py # Generate event sets
├── models/
│   ├── hydrology.py # Soil moisture, groundwater state
│   └── hydraulics.py # River level routing
```

- | — [stormgauge.py](#) # Gauge response orchestration
- | — [antecedent.py](#) # Antecedent conditions model
- | — utils/
 - | — [validation.py](#) # Sequence validation checks
 - | — [statistics.py](#) # Aggregate statistics calculation
 - | — [serialization.py](#) # JSON save/load
- | — tests/
 - | — test_duration_sampler.py
 - | — test_gap_sampler.py
 - | — test_doublet_generator.py
 - | — test_hydrology.py
 - | — test_integration.py

7.3 File Organization and Output

input/thames/

- | — doublet_sequences.json # 10-12 MB (10K doublets)
- | — sequences_summary.json # 200 KB (metadata only)
- | — gauge_responses.json # 20-30 MB (168-hour hydrographs)
- | — validation_report.json # Statistics and checks

JSON schema version: 2.0-doublet

8. Testing and Validation

8.1 Unit Tests

1. **Duration sampling** - Sample 1000 durations per category, verify min/mode/max bounds
2. **Gap sampling** - Verify gap distributions match INTER_STORM_GAP_PARAMS
3. **Timing constraint** - All generated sequences have Storm 2 end $\leq$ hour 156
4. **Drainage window** - All sequences have ≥ 12 hours precipitation-free at end
5. **Sequence validation** - num_storms = 2, len(storms) = 2, temporal ordering correct
6. **Aggregate statistics** - Recalculate and verify total precipitation, max/avg intensity

7. **JSON serialization** - Roundtrip test (save → load → verify equality)

8.2 Integration Tests

1. Generate 1000 test doublets across all intensity categories
2. Pass through gauge response model for all gauges
3. Verify hydrographs show:
 - Rise during Storm 1
 - Partial decline during gap
 - Rise during Storm 2 (to higher peak than Storm 1)
 - Final decline during drainage window (156-168h)
4. Check for compounding: `peak_2 > peak_1` in $\geq 60\%$ of sequences
5. Verify no hard-coded time windows or array size limits

8.3 Historical Validation

Target events:

- Winter 2013/14 Thames floods (multiple storms over weeks)
- Storm Henk 2024 (single major event for comparison)
- 2000 floods (multi-storm autumn sequence)
- 2003 floods (summer convective sequences)

Validation approach:

1. Extract doublet characteristics from historical events (duration, gap, peak ratios)
2. Generate 10,000 synthetic doublets
3. Compare distributions:
 - Storm durations within $\pm 30\%$ of historical mean
 - Gap durations within $\pm 30\%$ of historical mean
 - Peak 2 / Peak 1 ratios within 90% confidence interval
 - Total precipitation within $\pm 20\%$ of historical events
4. Statistical tests: Kolmogorov-Smirnov test ($\alpha = 0.05$)

Success criteria:

- Historical 1-in-100 year doublet events fall within synthetic 50-200 year range
- Storm count, duration, gaps within $\pm 30\%$ of synthetic mean

- Peak amplification (Storm 2 vs Storm 1) matches observed ratios

8.4 Performance Targets

Generation speed: <2 minutes for 10,000 doublet sequences with full gauge responses

Optimization strategies:

1. Parallel gauge response calculation (4× speedup on 4-core CPU)
2. Batch processing (1000 sequences per batch)
3. Caching of intensity/duration samples
4. Vectorized hydrology state updates (NumPy operations)

9. Backward Compatibility

9.1 Schema Versioning

Version	Description
v1.0	Original implementation (isolated storms, max 120h)
v1.1	Extended durations (Phase 1, max 240h)
v2.0-doublet	Two-storm sequences with drainage windows

Table 2: JSON schema version history

9.2 Requirements

1. Load v1.0 storm files (isolated storms) and convert to doublet-equivalent format
2. Maintain ability to generate v1.0-style output if requested
3. Generator metadata includes schema version and generation parameters

9.3 Migration Path

For users with existing v1.0 storm files:

```
def convert_v1_to_v2_doublet(v1_storms: List[StormScenario]) ->
List[StormSequence]:
```

```
"""
```

```
Convert isolated v1.0 storms to v2.0 doublet sequences.  
Each isolated storm becomes a sequence with 1 storm.
```

```
"""
```

```
sequences = []  
for storm in v1_storms:  
    seq = StormSequence(  
        sequence_id=f"SEQ-{uuid4()}",  
        sequence_type='isolated', # Not a true doublet  
        storms=[storm],  
        num_storms=1,  
        inter_storm_gap_hours=0,  
        # ... other fields  
    )  
    sequences.append(seq)  
return sequences
```

10. Risk Management

10.1 Identified Risks

Risk	Likelihood	Impact
Gauge response model doesn't handle long durations	Low	High
Multi-storm sequences break gauge response	Medium	High
Timing constraint too restrictive (few valid doublets)	Medium	Medium
File sizes exceed 50 MB	Low	Medium
Performance degrades with 10K sequences	Medium	Medium
Synthetic doublets don't match historical patterns	Medium	High
Backward compatibility broken	Low	High

Table 3: Risk assessment

10.2 Mitigation Strategies

- **Long durations:** Code inspection of stormgauge.py, extend time windows/arrays as needed
 - **Multi-storm gauge response:** Implement continuous state evolution (Section 5.2)
 - **Timing constraint:** If <95% of attempts succeed, relax MIN_DRAINAGE_WINDOW_HOURS to 8 hours
 - **File sizes:** Monitor output, compress JSON if needed, split into multiple files
 - **Performance:** Profile code, optimize bottlenecks, implement parallel processing
 - **Historical validation:** Extensive comparison to 2013/14, 2000, 2003 events; tune parameters if needed
 - **Backward compatibility:** Comprehensive migration tests, version tagging in all outputs
-

11. Implementation Roadmap

11.1 Phase 1: Duration Extension (1-2 days)

1. Update DURATION_PARAMS in storm.py
2. Verify StormScenario dataclass accepts arbitrary durations
3. Test stormgauge.py with 240-hour storms
4. Update metadata and schema version to v1.1
5. Generate 10,000 test storms across all categories
6. Validate duration distributions

11.2 Phase 2: Doublet Framework (1 week)

1. Implement StormSequence dataclass
2. Create generate_doublet_sequence() function (modular helpers)
3. Implement timing constraint logic ($\leq 156h$, $\geq 12h$ drainage)
4. Create generate_event_set() for batch generation
5. Implement gap sampling (INTER_STORM_GAP_PARAMS)
6. Add intensity correlation (Storm 2 typically $\geq$ Storm 1)
7. Update schema version to v2.0-doublet
8. Unit tests for all components

11.3 Phase 3: Gauge Response Integration (3-5 days)

1. Implement continuous state evolution in stormgauge.py
2. Add simplified antecedent conditions model (Section 5.3)
3. Compute gauge responses for doublet sequences
4. Verify compounding behavior (Storm 2 peak > Storm 1 peak)
5. Verify drainage during gaps and final window
6. Integration tests with 1000 doublets

11.4 Phase 4: Validation and Production (1 week)

1. Historical validation against 2013/14, 2000, 2003 events
2. Statistical tests (KS test, chi-square)
3. Generate production 10,000-doublet catalog
4. Performance profiling and optimization
5. Documentation (README, parameter reference, user guide)
6. Generate validation report

11.5 Future Enhancements

- Full state-space antecedent conditions model (Section 6.1)
- Duration-intensity copula (Section 6.2)
- Tidal signal injection (Section 5.4)
- Bayesian parameter uncertainty quantification
- Extension to cluster (3-4 storms) and persistent (4-5 storms) sequences

12. Document Control

Version: 2.0-doublet

Status: SPECIFICATION - Ready for Implementation

Date: March 5, 2026

Author: Technical Specification Team

Approved By: David Kelly

Change Log:

- v1.0 (Feb 26, 2026) - Original multi-storm specification
- v2.0-doublet (Mar 5, 2026) - Focused on two-storm sequences, added drainage window constraint, emphasized modularity, added hydrologic-hydraulic integration details

Next Steps:

1. Review and approve this focused specification
 2. Begin Phase 1 implementation (duration extension)
 3. Implement Phase 2 (doublet framework with modular code)
 4. Validate against Thames historical doublet events
 5. Generate production 10K doublet catalog
-

References

- [1] Original Storm Generator Implementation Spec v1.0. (2026, February 26). Thames Valley Physical Risk Swaps project documentation.
- [2] Salvadori, G., & De Michele, C. (2004). Frequency analysis via copulas: Theoretical aspects and applications to hydrological events. *Water Resources Research*, 40(12).
- [3] Serinaldi, F., & Grimaldi, S. (2023). A Framework of Dependence Modeling and Evaluation System for Compound Flooding. *Water Resources Research*, 59(7).
- [4] Dupas, R., et al. (2015). Clustering hydrological events using dynamic time warping. *Journal of Hydrology*, 593, 125802.
- [5] Brunner, M. I., et al. (2021). Stochastic simulation of streamflow and spatial extremes: A continuous, wavelet-based approach. *Hydrology and Earth System Sciences*, 25, 5153-5174.
- [6] Environment Agency. (2024). Thames flooding January 2024 - Section 19 investigation report. *UK Government*.
- [7] Port of London Authority. (2024). Tidal information and tide tables. <https://pla.co.uk/tidal-information>

Storm Generator v2.0

Implementation Specification

Thames Valley Physical Risk Swaps - Enhanced Multi-Storm Sequences

Project: PRS Platform Storm Generator v2.0

Date: March 5, 2026

Prepared for: David Kelly

Scope: Single-Scenario Two-Storm Implementation (Doublet Focus)

Version: 2.0-doublet-focused

Executive Summary

This specification provides detailed implementation guidance for a focused storm generation system targeting **single-scenario, two-storm (doublet) sequences** with explicit inter-storm drainage periods to model compound flooding events in the Thames Valley.

Key Innovation

Moving from independent storm sampling to **temporally-correlated doublet sequences** where:

- Each scenario contains exactly **two heavy precipitation events**
- Clear **inter-storm gaps** (24–96 hours) allow observable river level decline
- All precipitation stops **≥12 hours before hour 168** to show final drainage
- Antecedent soil moisture from Storm 1 amplifies Storm 2 response
- Hydraulic and hydrologic models work together to capture compounding effects

Success Criteria

- Generate 10,000 realistic two-storm flood scenarios
 - Respect 168-hour insurance event windows
 - Show clear water level decline between precipitation pulses
 - Model soil moisture persistence and compounding effects
 - Maintain market-calibrated intensity distribution (20-60 bps fair spreads)
 - File sizes remain manageable (<50 MB for 10K events)
 - Performance: <2 minutes to generate 10K doublet scenarios with gauge responses
-

1. Project Context and Constraints

1.1 Document Purpose

This is a **specification document**, not implementation code. It describes:

- What needs to be built
- How components should interact
- Data structures and schemas
- Algorithms and logic flows
- Integration points with existing code
- Testing and validation criteria
- Mathematical foundations for hydrologic-hydraulic coupling

1.2 Project Constraints

Scenario structure: Single scenario type (doublet) with exactly 2 storms per sequence

Event count: 10,000 doublet scenarios maximum

Event window: 168 hours (7 days) per insurance industry standard

Drainage requirement: Final 12 hours (156–168h) must be precipitation-free

Gauge data: DEFRA gauges are real, others extrapolated

Climate scenarios: NOT in scope (present-day conditions only)

Backward compatibility: Must maintain ability to load existing storm files

1.3 Known Limitations

Validation data: Limited historical gauge data availability for doublet sequences

Antecedent conditions: Initial implementation uses simplified state model; full state-space model (Section 6) is future enhancement

Tidal influence: Current gauge levels may show tidal undulations; tide schedule injection capability to be added if needed

2. Core Architecture: Doublet-Focused Storm Sequences

2.1 Overview

Each "scenario" is a **StormSequence** object containing exactly two **StormScenario** objects (a doublet), separated by an inter-storm gap that allows partial drainage. The entire sequence fits within a 168-hour event window, with all precipitation ending by hour 156.

2.2 Data Structures

2.2.1 StormScenario (Individual Storm)

```
@dataclass
```

```
class StormScenario:
```

```
    """
```

```
    A single precipitation event within a sequence.
```

```
    """
```

```
    # Identity
```

```
    storm_id: str # Format: "STORM-{uuid4}"
```

```
    scenario_id: str # Parent sequence ID
```

```
    # Temporal
```

```
    start_time: str      # ISO format, hours from sequence start
```

```
    duration_hours: int  # Storm duration
```

```
    # Intensity
```

```
    intensity_category: str # 'moderate', 'severe', 'extreme', 'catastrophic'
```

```
    intensity_factor: float # Multiplier for base intensity
```

```
precipitation_mm: float # Total precipitation
```

```
# Metadata
```

```
generated_at: str
```

```
catchment_id: str
```

2.2.2 StormSequence (Doublet Container)

```
@dataclass
```

```
class StormSequence:
```

```
"""
```

A doublet sequence: exactly two storms treated as one flood event.
Insurance industry 'hours clause' implementation.

```
"""
```

```
# Identity
```

```
sequence_id: str # Format: "SEQ-{uuid4}"
```

```
sequence_type: str # Always 'doublet' for this implementation
```

```
# Temporal structure
```

```
sequence_start: str # ISO format
```

```
sequence_end: str # Must be  $\leq 156$  hours from start
```

```
total_duration_hours: int # Actual duration of precip + gap
```

```
event_window_hours: int # Always 168
```

```
drainage_window_hours: int # Final dry period ( $\geq 12$  hours)
```

```
# Storm composition
```

```
storms: List[StormScenario] # Always exactly 2 storms
```

```
num_storms: int # Always 2
```

```
# Temporal gaps
```

```
inter_storm_gap_hours: int # Gap between storm 1 end and storm 2 start
```

```
# Aggregate statistics
```

```
total_precipitation_mm: float
```

```
max_intensity_mm: float
```

```
avg_intensity_mm: float
```

```
cumulative_intensity_factor: float
```

```
# Antecedent conditions (for gauge response)
antecedent_soil_moisture: Optional[str] = None    # 'dry', 'normal', 'wet'
antecedent_groundwater: Optional[str] = None    # 'low', 'normal', 'high'

# Metadata
generated_at: str
catchment_id: str
```

Validation rules:

- num_storms must equal 2
- len(storms) must equal 2
- sequence_type must be 'doublet'
- sequence_end ≤ 156 hours (leaves ≥12h drainage)
- total_duration_hours = sum(storm durations) + inter_storm_gap_hours
- storms[0].start_time < storms[1].start_time

3. Storm Generation Parameters

3.1 Duration Parameters (Phase 1 - Extended Durations)

Updated from original spec to support up to 240-hour maximum durations for individual storms within sequences.

```
DURATION_PARAMS = {
# (min, mode, max) hours - triangular distribution
'moderate': (12, 30, 60), # 2.5 day maximum
'severe': (24, 48, 96), # 4 day maximum
'extreme': (36, 72, 144), # 6 day maximum
'catastrophic': (48, 120, 240), # 10 day maximum
}
```

Key design decisions:

1. Triangular distribution preserved - maintains statistical properties
2. Each tier approximately doubles previous maximum

3. Mode shifted up - reflects that longer events are more common than previously assumed
4. Conservative extreme tail - catastrophic events get full 10-day capability

3.2 Inter-Storm Gap Parameters

Gaps must be long enough to show observable river level decline between storms.

```
INTER_STORM_GAP_PARAMS = {  
# (min, mode, max) hours - triangular distribution  
'medium': (24, 48, 72), # Partial drainage  
'long': (48, 96, 144), # Substantial drainage  
}
```

Gap type selection: For doublet sequences, default to 'medium' gaps (24–72 hours), with option to use 'long' gaps for more pronounced drainage effects.

Physical interpretation:

- **Medium gaps (24-72h):** River levels drop 30-60% from Storm 1 peak before Storm 2 begins
- **Long gaps (48-144h):** River levels drop 60-80% from Storm 1 peak, approaching baseline

3.3 Intensity Parameters

Storm intensities vary around base intensity to create realistic variability within doublets.

```
INTENSITY_VARIATION = 0.20 # ±20% around base intensity
```

Probability that first storm is most intense

```
FIRST_STORM_DOMINANT_PROB = 0.30 # 30%
```


Otherwise second storm is equal or more intense (70%)

Rationale: In compound flooding, the second storm often produces higher peaks due to antecedent wetness, even if precipitation intensity is similar or slightly lower.

3.4 Drainage Window Constraint

EVENT_WINDOW_HOURS = 168

MIN_DRAINAGE_WINDOW_HOURS = 12 # Precipitation-free period at end

Constraint: $\text{sequence_end} \leq \text{EVENT_WINDOW_HOURS} - \text{MIN_DRAINAGE_WINDOW_HOURS}$

MAX_PRECIP_END_HOUR = 156

4. Doublet Generation Algorithm

4.1 High-Level Flow

1. Sample intensity category for the doublet
2. Generate Storm 1 (duration, intensity)
3. Sample inter-storm gap duration
4. Generate Storm 2 (duration, intensity, conditioned on Storm 1)
5. Verify timing constraint: Storm 2 end $\leq$ hour 156
6. If constraint violated, resample durations/gap until satisfied
7. Create StormSequence object
8. Calculate aggregate statistics

4.2 Detailed Algorithm

```
def generate_doublet_sequence(intensity_category: str,
    catchment_id: str,
    gap_type: str = 'medium') -> StormSequence:
    """
```

Generate a single doublet sequence with two storms.

Returns:

StormSequence with exactly 2 storms, ending by hour 156
"""

```
sequence_id = f"SEQ-{uuid4()}"
```

```
# Step 1: Sample base intensity
```

```
base_intensity = sample_intensity_factor(intensity_category)
```

```
# Step 2: Generate Storm 1
```

```
duration_1 = sample_duration(intensity_category)
```

```
intensity_1 = base_intensity * (1 + uniform(-0.2, 0.2))
```

```
storm_1 = StormScenario(
```

```
    storm_id=f"STORM-{uuid4()}",
```

```
    scenario_id=sequence_id,
```

```
    start_time="0h", # Sequence starts at hour 0
```

```
    duration_hours=duration_1,
```

```
    intensity_category=intensity_category,
```

```
    intensity_factor=intensity_1,
```

```
    precipitation_mm=calculate_precipitation(duration_1, intensity_1),
```

```
    generated_at=datetime.now().isoformat(),
```

```
    catchment_id=catchment_id
```

```
)
```

```
# Step 3: Sample inter-storm gap
```

```
gap_hours = sample_gap(gap_type)
```

```
# Step 4: Generate Storm 2
```

```
storm_2_start = duration_1 + gap_hours
```

```
duration_2 = sample_duration(intensity_category)
```

```

# Intensity correlation: 70% chance Storm 2 ≥ Storm 1
if random() < 0.70:
    intensity_2 = base_intensity * (1 + uniform(0.0, 0.2))
else:
    intensity_2 = base_intensity * (1 + uniform(-0.2, 0.0))

storm_2 = StormScenario(
    storm_id=f"STORM-{uuid4()}",
    scenario_id=sequence_id,
    start_time=f"{storm_2_start}h",
    duration_hours=duration_2,
    intensity_category=intensity_category,
    intensity_factor=intensity_2,
    precipitation_mm=calculate_precipitation(duration_2, intensity_2),
    generated_at=datetime.now().isoformat(),
    catchment_id=catchment_id
)

# Step 5: Verify timing constraint
storm_2_end = storm_2_start + duration_2

if storm_2_end > MAX_PRECIP_END_HOUR:
    # Resample with shorter durations or gap
    return generate_doublet_sequence(
        intensity_category, catchment_id, gap_type
    )

# Step 6: Create sequence object
sequence = StormSequence(
    sequence_id=sequence_id,
    sequence_type='doublet',
    sequence_start=datetime.now().isoformat(),
    sequence_end=f"{storm_2_end}h",
    total_duration_hours=storm_2_end,
    event_window_hours=168,
    drainage_window_hours=168 - storm_2_end,
    storms=[storm_1, storm_2],

```

```

        num_storms=2,
        inter_storm_gap_hours=gap_hours,
        total_precipitation_mm=storm_1.precipitation_mm + storm_2.precipitation_mm,
        max_intensity_mm=max(storm_1.intensity_factor, storm_2.intensity_factor),
        avg_intensity_mm=(storm_1.intensity_factor + storm_2.intensity_factor) / 2,
        cumulative_intensity_factor=storm_1.intensity_factor + storm_2.intensity_factor,
        antecedent_soil_moisture='normal', # Initial state
        antecedent_groundwater='normal',
        generated_at=datetime.now().isoformat(),
        catchment_id=catchment_id
    )

    return sequence

```

4.3 Batch Generation

```

def generate_event_set(count: int = 10000,
    intensity_weights: dict = None,
    gap_type: str = 'medium') -> List[StormSequence]:
    """

```

Generate a set of doublet sequences.

Args:

count: Number of doublet sequences to generate
 intensity_weights: Dict of {category: probability}
 gap_type: 'medium' or 'long'

Returns:

List of StormSequence objects

"""

if intensity_weights is None:

```

    intensity_weights = {
        'moderate': 0.40,
        'severe': 0.35,
        'extreme': 0.20,
        'catastrophic': 0.05
    }

```

```
sequences = []
for i in range(count):
    category = sample_intensity_category(intensity_weights)
    seq = generate_doublet_sequence(
        intensity_category=category,
        catchment_id='thames',
        gap_type=gap_type
    )
    sequences.append(seq)

return sequences
```

5. Hydrologic-Hydraulic Integration

5.1 Gauge Response Model

The gauge response model (`stormgauge.py`) transforms precipitation time series into river level (or flow) hydrographs at each gauge location.

Key capabilities:

- Time-series based convolution with lag and decay parameters
- Handles arbitrary storm durations (tested up to 240 hours)
- Configurable response kernels per gauge/catchment
- Supports antecedent moisture state amplification

5.2 Compounding Between Storms

For doublet sequences, gauge responses must capture compounding:

Storm 1:

- Rises from baseline river level
- Peak occurs at `storm_1_end + lag_hours`
- Begins recession during gap period

Gap period:

- Partial drainage (river level drops but does NOT return to baseline)
- Soil moisture remains elevated
- Groundwater storage decays slowly

Storm 2:

- Rises from elevated baseline (not from original baseline)
- Produces higher peak than Storm 1 of equal intensity would produce
- Final recession extends into drainage window (hours 156–168)

Implementation approach:

```
def compute_sequence_gauge_response(sequence: StormSequence,
gauge_id: str,
gauge_params: dict) -> np.ndarray:
    """
```

Compute continuous gauge response for entire doublet sequence.

Returns:

Array of length 168 (hourly levels for full event window)

"""

Initialize state

soil_moisture = initial_soil_moisture()

groundwater = initial_groundwater()

river_level = baseline_level(gauge_id)

levels = np.zeros(168)

for hour in range(168):

Check if precipitation is occurring

precip = get_precipitation_at_hour(sequence, hour)

Update hydrologic state

soil_moisture, groundwater, quickflow = update_hydrology(

soil_moisture, groundwater, precip, hour

)

```

# Update river level (hydraulic routing)
river_level = update_hydraulics(
    river_level, quickflow, gauge_params
)

levels[hour] = river_level

return levels

```

5.3 Antecedent Conditions State Model

Simplified state evolution for initial implementation (see Section 6 for full mathematical model):

Soil moisture evolution:

$$S_{t+1} = S_t + \alpha_P \cdot P_t - \beta_S \cdot S_t \quad (9)$$

where:

- S_t = soil moisture at hour t
- P_t = precipitation at hour t
- α_P = infiltration coefficient (0.6–0.8)
- β_S = drainage coefficient (0.01–0.05 per hour)

Groundwater evolution:

$$G_{t+1} = G_t + \beta_S \cdot S_t - \gamma_G \cdot G_t \quad (10)$$

where:

- G_t = groundwater storage at hour t
- γ_G = baseflow coefficient (0.005–0.02 per hour)

Runoff amplification:

$$Q_t = (1 + \lambda_S \cdot S_t + \lambda_G \cdot G_t) \cdot P_t \quad (11)$$

where:

- Q_t = quickflow/runoff at hour t
- λ_S, λ_G = amplification coefficients (calibrated from gauge data)

Practical benefits:

- Models why Storm 2 produces higher peaks than Storm 1 of equal intensity
- Explains 168-hour event window (time for partial drainage)
- Provides mechanistic basis for gap duration effects
- Shows observable level decline during gaps

5.4 Tidal Signal Injection (Optional)

If observed gauge levels show tidal undulations not captured by the hydrologic model, add a separate tidal component:

River-only level: $R(t)$ from storm generator and gauge response model

Tidal level: $T(t)$ from external tide predictions or parametric model

Total gauge level: $L(t) = R(t) + \alpha \cdot T(t)$

where α is a scaling factor (0–1) based on gauge distance from estuary.

Tidal model options:

1. **Data-driven:** Use actual tide predictions for London Bridge, interpolate and repeat for 168-hour windows
2. **Analytical:** Approximate semi-diurnal tide with sum of sinusoids (M2, S2 components)

Implementation note: Compute drainage on $R(t)$ alone (hydrologic recession), then add $T(t)$ afterwards so tides modulate observed level without affecting catchment storage dynamics.

6. Mathematical Foundations

6.1 State-Space Model for Antecedent Conditions (Future Enhancement)

For a more rigorous treatment of compounding effects, implement a full state-space model:

State evolution:

$$S_{t+1} = f_S(S_t, G_t, P_t, \Delta t) + \epsilon_S \quad (12)$$

$$G_{t+1} = f_G(S_t, G_t, \Delta t) + \epsilon_G \quad (13)$$

where:

- f_S, f_G = nonlinear state transition functions
- ϵ_S, ϵ_G = stochastic components (process noise)

Observation model:

$$L_t = h(R_t, S_t, G_t) + \nu_t \quad (14)$$

where:

- h = observation function (rating curve)
- ν_t = measurement noise

Calibration approach:

1. Use historical Thames doublet events (2013/14, 2000, 2003 floods)
2. Fit state-space model using Extended Kalman Filter or Particle Filter
3. Estimate amplification coefficients λ_S, λ_G via maximum likelihood
4. Validate that synthetic doublets match observed peak ratios (Storm 2 / Storm 1)

6.2 Duration-Intensity Correlation

Physically, long-duration storms tend to be lower intensity (stratiform), while short convective storms are high intensity. Model this using a Gaussian copula with negative correlation:

$$\rho_{DI} = -0.3 \quad (15)$$

Practical sampling:

1. Sample (U_D, U_I) from Gaussian copula with correlation ρ_{DI}
2. Transform: $D = F_D^{-1}(U_D), I = F_I^{-1}(U_I)$
3. Result: Long storms $\rightarrow$ lower intensity, short storms $\rightarrow$ higher intensity

where F_D, F_I are triangular CDFs for duration and intensity.

6.3 Gap Duration and Drainage Efficiency

The relationship between gap duration and river level recovery can be modeled empirically:

$$\frac{L_{\text{gap_end}} - L_{\text{baseline}}}{L_{\text{storm1_peak}} - L_{\text{baseline}}} = \exp\left(-\frac{\text{gap_hours}}{\tau_{\text{recession}}}\right) \quad (16)$$

where $\tau_{\text{recession}}$ is the recession timescale (typically 24–72 hours for Thames catchment).

Calibration: Fit $\tau_{\text{recession}}$ from historical hydrographs to ensure modeled drainage matches observed behavior.

7. Implementation Architecture and Modularity

7.1 Code Structure Principles

CRITICAL REQUIREMENT: The storm generator, gauge response model, and related utilities must be implemented as highly modular components to support maintenance, testing, and future extensions.

Modularity guidelines:

- Functions should be cohesive and focused on a single responsibility
- **Target: ≤ 300 lines per module or function**
- **Hard maximum: 450 lines per module or function**
- Where logic exceeds this, decompose into smaller helpers

Example decomposition:

Instead of one large `generate_and_process_sequences()` function, split into:

- `sample_duration(category)` - 20 lines
- `sample_gap(gap_type)` - 15 lines
- `generate_storm(params)` - 50 lines

- `generate_doublet_sequence(params)` - 100 lines
- `compute_gauge_response(sequence, gauge)` - 80 lines
- `update_hydrology(state, precip)` - 60 lines
- `update_hydraulics(level, inflow)` - 40 lines

Benefits:

1. Easier debugging (isolate failures to specific functions)
2. Targeted unit tests (test `sample_gap` independently)
3. Reduced regression risk when adding enhancements
4. Clearer code review and approval process
5. Simpler integration of mathematical enhancements (Section 6)

7.2 Suggested Module Structure

```
port/src/
├── storm.py # Core StormScenario dataclass
├── storm_sequence.py # StormSequence dataclass
├── generators/
│   ├── duration_sampler.py # Sample storm durations
│   ├── gap_sampler.py # Sample inter-storm gaps
│   ├── intensity_sampler.py # Sample intensities
│   ├── doublet_generator.py # Generate doublet sequences
│   └── batch_generator.py # Generate event sets
├── models/
│   ├── hydrology.py # Soil moisture, groundwater state
│   ├── hydraulics.py # River level routing
│   ├── stormgauge.py # Gauge response orchestration
│   └── antecedent.py # Antecedent conditions model
├── utils/
│   ├── validation.py # Sequence validation checks
│   ├── statistics.py # Aggregate statistics calculation
│   └── serialization.py # JSON save/load
├── tests/
├── test_duration_sampler.py
├── test_gap_sampler.py
├── test_doublet_generator.py
├── test_hydrology.py
└── test_integration.py
```

7.3 File Organization and Output

input/thames/

- |— doublet_sequences.json # 10-12 MB (10K doublets)
- |— sequences_summary.json # 200 KB (metadata only)
- |— gauge_responses.json # 20-30 MB (168-hour hydrographs)
- |— validation_report.json # Statistics and checks

JSON schema version: 2.0-doublet

8. Testing and Validation

8.1 Unit Tests

1. **Duration sampling** - Sample 1000 durations per category, verify min/mode/max bounds
2. **Gap sampling** - Verify gap distributions match INTER_STORM_GAP_PARAMS
3. **Timing constraint** - All generated sequences have Storm 2 end $\leq$ hour 156
4. **Drainage window** - All sequences have ≥ 12 hours precipitation-free at end
5. **Sequence validation** - num_storms = 2, len(storms) = 2, temporal ordering correct
6. **Aggregate statistics** - Recalculate and verify total precipitation, max/avg intensity
7. **JSON serialization** - Roundtrip test (save $\rightarrow$ load $\rightarrow$ verify equality)

8.2 Integration Tests

1. Generate 1000 test doublets across all intensity categories
2. Pass through gauge response model for all gauges
3. Verify hydrographs show:
 - Rise during Storm 1
 - Partial decline during gap
 - Rise during Storm 2 (to higher peak than Storm 1)
 - Final decline during drainage window (156-168h)
4. Check for compounding: peak_2 > peak_1 in $\geq 60\%$ of sequences
5. Verify no hard-coded time windows or array size limits

8.3 Historical Validation

Target events:

- Winter 2013/14 Thames floods (multiple storms over weeks)
- Storm Henk 2024 (single major event for comparison)
- 2000 floods (multi-storm autumn sequence)
- 2003 floods (summer convective sequences)

Validation approach:

1. Extract doublet characteristics from historical events (duration, gap, peak ratios)
2. Generate 10,000 synthetic doublets
3. Compare distributions:
 - Storm durations within $\pm 30\%$ of historical mean
 - Gap durations within $\pm 30\%$ of historical mean
 - Peak 2 / Peak 1 ratios within 90% confidence interval
 - Total precipitation within $\pm 20\%$ of historical events
4. Statistical tests: Kolmogorov-Smirnov test ($\alpha = 0.05$)

Success criteria:

- Historical 1-in-100 year doublet events fall within synthetic 50-200 year range
- Storm count, duration, gaps within $\pm 30\%$ of synthetic mean
- Peak amplification (Storm 2 vs Storm 1) matches observed ratios

8.4 Performance Targets

Generation speed: <2 minutes for 10,000 doublet sequences with full gauge responses

Optimization strategies:

1. Parallel gauge response calculation (4× speedup on 4-core CPU)
 2. Batch processing (1000 sequences per batch)
 3. Caching of intensity/duration samples
 4. Vectorized hydrology state updates (NumPy operations)
-

9. Backward Compatibility

9.1 Schema Versioning

Version	Description
v1.0	Original implementation (isolated storms, max 120h)
v1.1	Extended durations (Phase 1, max 240h)
v2.0-doublet	Two-storm sequences with drainage windows

Table 4: JSON schema version history

9.2 Requirements

1. Load v1.0 storm files (isolated storms) and convert to doublet-equivalent format
2. Maintain ability to generate v1.0-style output if requested
3. Generator metadata includes schema version and generation parameters

9.3 Migration Path

For users with existing v1.0 storm files:

```
def convert_v1_to_v2_doublet(v1_storms: List[StormScenario]) ->
List[StormSequence]:
    """
```

```
    Convert isolated v1.0 storms to v2.0 doublet sequences.
    Each isolated storm becomes a sequence with 1 storm.
    """
```

```
    sequences = []
    for storm in v1_storms:
        seq = StormSequence(
            sequence_id=f"SEQ-{uuid4()}",
            sequence_type='isolated', # Not a true doublet
            storms=[storm],
            num_storms=1,
            inter_storm_gap_hours=0,
            # ... other fields
```

```
)  
sequences.append(seq)  
return sequences
```

10. Risk Management

10.1 Identified Risks

Risk	Likelihood	Impact
Gauge response model doesn't handle long durations	Low	High
Multi-storm sequences break gauge response	Medium	High
Timing constraint too restrictive (few valid doublets)	Medium	Medium
File sizes exceed 50 MB	Low	Medium
Performance degrades with 10K sequences	Medium	Medium
Synthetic doublets don't match historical patterns	Medium	High
Backward compatibility broken	Low	High

Table 5: Risk assessment

10.2 Mitigation Strategies

- **Long durations:** Code inspection of `stormgauge.py`, extend time windows/arrays as needed
- **Multi-storm gauge response:** Implement continuous state evolution (Section 5.2)
- **Timing constraint:** If <95% of attempts succeed, relax `MIN_DRAINAGE_WINDOW_HOURS` to 8 hours
- **File sizes:** Monitor output, compress JSON if needed, split into multiple files
- **Performance:** Profile code, optimize bottlenecks, implement parallel processing
- **Historical validation:** Extensive comparison to 2013/14, 2000, 2003 events; tune parameters if needed

- **Backward compatibility:** Comprehensive migration tests, version tagging in all outputs
-

11. Implementation Roadmap

11.1 Phase 1: Duration Extension (1-2 days)

1. Update DURATION_PARAMS in storm.py
2. Verify StormScenario dataclass accepts arbitrary durations
3. Test stormgauge.py with 240-hour storms
4. Update metadata and schema version to v1.1
5. Generate 10,000 test storms across all categories
6. Validate duration distributions

11.2 Phase 2: Doublet Framework (1 week)

1. Implement StormSequence dataclass
2. Create generate_doublet_sequence() function (modular helpers)
3. Implement timing constraint logic ($\leq 156h$, $\geq 12h$ drainage)
4. Create generate_event_set() for batch generation
5. Implement gap sampling (INTER_STORM_GAP_PARAMS)
6. Add intensity correlation (Storm 2 typically $\geq$ Storm 1)
7. Update schema version to v2.0-doublet
8. Unit tests for all components

11.3 Phase 3: Gauge Response Integration (3-5 days)

1. Implement continuous state evolution in stormgauge.py
2. Add simplified antecedent conditions model (Section 5.3)
3. Compute gauge responses for doublet sequences
4. Verify compounding behavior (Storm 2 peak $>$ Storm 1 peak)
5. Verify drainage during gaps and final window
6. Integration tests with 1000 doublets

11.4 Phase 4: Validation and Production (1 week)

1. Historical validation against 2013/14, 2000, 2003 events
2. Statistical tests (KS test, chi-square)
3. Generate production 10,000-doublet catalog
4. Performance profiling and optimization
5. Documentation (README, parameter reference, user guide)

6. Generate validation report

11.5 Future Enhancements

- Full state-space antecedent conditions model (Section 6.1)
- Duration-intensity copula (Section 6.2)
- Tidal signal injection (Section 5.4)
- Bayesian parameter uncertainty quantification
- Extension to cluster (3-4 storms) and persistent (4-5 storms) sequences

12. Document Control

Version: 2.0-doublet

Status: SPECIFICATION - Ready for Implementation

Date: March 5, 2026

Author: Technical Specification Team

Approved By: David Kelly

Change Log:

- v1.0 (Feb 26, 2026) - Original multi-storm specification
- v2.0-doublet (Mar 5, 2026) - Focused on two-storm sequences, added drainage window constraint, emphasized modularity, added hydrologic-hydraulic integration details

Next Steps:

1. Review and approve this focused specification
2. Begin Phase 1 implementation (duration extension)
3. Implement Phase 2 (doublet framework with modular code)
4. Validate against Thames historical doublet events
5. Generate production 10K doublet catalog

References

[1] Original Storm Generator Implementation Spec v1.0. (2026, February 26). Thames Valley Physical Risk Swaps project documentation.

- [2] Salvadori, G., & De Michele, C. (2004). Frequency analysis via copulas: Theoretical aspects and applications to hydrological events. *Water Resources Research*, 40(12).
- [3] Serinaldi, F., & Grimaldi, S. (2023). A Framework of Dependence Modeling and Evaluation System for Compound Flooding. *Water Resources Research*, 59(7).
- [4] Dupas, R., et al. (2015). Clustering hydrological events using dynamic time warping. *Journal of Hydrology*, 593, 125802.
- [5] Brunner, M. I., et al. (2021). Stochastic simulation of streamflow and spatial extremes: A continuous, wavelet-based approach. *Hydrology and Earth System Sciences*, 25, 5153-5174.
- [6] Environment Agency. (2024). Thames flooding January 2024 - Section 19 investigation report. *UK Government*.
- [7] Port of London Authority. (2024). Tidal information and tide tables. <https://pla.co.uk/tidal-information>